FORO EVALUABLE RESPUESTAS: LUCAS TAMARA TREJO

PREGUNTA 1: Que es un proceso estocástico, ¿qué tipos hay y que aplicaciones tienen?

En la teoría de la probabilidad un proceso estocástico es un concepto matemático que sirve para características una sucesión de variables aleatorias que evolucionan en funciona de otra variable en general el tiempo.

Tiempo	Estados	¿Cómo entenderlo?	Ejemplo representativo
Discreto	**Discretos**	El proceso se observa por pasos (día 1, día 2, día 3) y solo existen opciones exactas o contables.	Clima: hoy = Soleado, mañana = Nublado, pasado mañana = Lluvioso (Cadena de Markov).
Discreto	**Continuos**	El proceso se observa por pasos, pero el valor puede ser cualquier número real (con decimales).	Precio del dólar cada día: 3.71, 3.74, 3.69...
Continuo	**Discretos**	El proceso puede cambiar en cualquier instante, pero los estados siguen siendo valores enteros o categorías.	Número de personas dentro de una tienda: 0, 1, 2, 3... (nunca 2.5 personas).
Continuo	**Continuos**	El proceso cambia continuamente y el estado también puede tomar cualquier valor real.	Temperatura medida constantemente: 24.31 °C, 24.318 °C, 24.320 °C...

- Web e informática: el PageRank de Google — modela a un navegante que hace clic al azar y ordena las páginas según la distribución estacionaria π que resuelve πP = π.
- Colas y redes de comunicación: desborde de buffers y el modelo M/M/1.
- Física: difusión de gases con el modelo de urnas de Ehrenfest.
- Finanzas y seguros: movimiento browniano, valoración de productos financieros y ruina de aseguradoras
- Epidemiología: propagación de epidemias
- Optimización y control: cadenas con coste y recompensa, base de los problemas de decisión.
- Origen histórico: Markov (1913) analizó 20.000 letras de un poema de Pushkin como cadena de dos estados, vocal/consonante.

PREGUNTA 2: De que tipo de proceso son las cadenas de Markov?

La cadenas de Markov son un tipo de proceso estocástico de tiempo discreto y con estados discretos, eso significa que avanza por pasos como ejemplo hoy mañana y pasado mañana por lo que puede encontrase con un conjunto de estados definidos y no en valores continuos con decimales, además una cadena de Markov no necesita analizar toda la información histórica para predecir el siguiente estado eso quiere decir que solo utiliza el estado presente para calcular el estado futuro.

$$P(X_{n+1} = j \mid X_n = i, X_{n-1}, ..., X_0) = P(X_{n+1} = j \mid X_n = i) = p_{ij}$$

PREGUNTA 3: ¿Que aplicaciones tienen los MDP en IA?
Un MDP extiende la cadena de Markov con acciones y recompensas: (S, A, P, R, γ).
El agente aprende una política π que maximiza $E[\Sigma \gamma^t R_{t+1}]$. Básicamente,
ese framework IS el reinforcement learning.

Aplicaciones concretas:
- Q-learning, DQN, PPO, SAC → todos resuelven MDPs
- Robótica: navegación autónoma, control de brazos
- AlphaGo / AlphaZero: MDP + MCTS + deep learning
- RLHF en LLMs (ChatGPT, Claude): el fine-tuning usa MDPs donde la acción es el siguiente token y la recompensa viene de preferencias humanas
- Medicina: optimización de protocolos de tratamiento
- Data centers: scheduling de recursos (Google lo usa en producción)

PREGUNTA 4: Que es una matriz estocástica y que aplicación tiene?

Una matriz estocástica es una matriz de probabilidades que describe la probabilidad de pasar de un estado a otro en una cadena de Markov. Todos sus elementos representan probabilidades, por lo que son mayores o iguales que cero y la suma de cada fila es igual a uno. Su principal aplicación es describir y calcular la evolución de un proceso de Markov, obtener las probabilidades de transición entre estados y determinar el estado estacionario del sistema.

PREGUNTA 5: Que es el vector de estado y que representa? El vector de estado en el instante n es el vector fila

$v_n = (P(X_n = 1), P(X_n = 2), ..., P(X_n = k))$

cuya componente i-ésima es la probabilidad de que el proceso esté en el estado i en ese instante. Sus entradas cumplen $v_n(i) \geq 0$ y $\Sigma_i\, v_n(i) = 1$, por lo que es una distribución de probabilidad sobre el espacio de estados.

Qué representa: no indica en qué estado está el sistema, sino cuán probable es cada estado. El caso de certeza absoluta es el vector $\delta_i = (0, ..., 1, ..., 0)$, con toda la probabilidad en un solo estado.

Evolución: se obtiene multiplicando por la matriz de transición, $v_n = v_0 \cdot P^n$. Si la cadena es ergódica, v_n converge a la distribución estacionaria π cuando $n \to \infty$, independientemente del vector inicial v_0.

PREGUNTA 6: ¿Qué es el estado estacionario? ¿Cómo se calcula? ¿Qué significa?

Es la distribución de probabilidad π que no cambia al aplicar la matriz de transición:

$\pi \cdot P = \pi$, con $\pi_i \geq 0$ y $\Sigma_i\, \pi_i = 1$

El PDF indica que "estacionario", "de equilibrio" y "estado estable" significan lo mismo.

Cómo se calcula. Se escribe el sistema $\pi = \pi P$ ecuación por ecuación (ecuaciones por la izquierda). Como una ecuación sobra, se reemplaza por la normalización $\Sigma_i\, \pi_i = 1$, y así π queda determinado de forma única.

Qué significa. A largo plazo la cadena converge a π, sin importar el estado inicial. Además, el tiempo medio de retorno al estado i es $m_i = 1/\pi_i$, y π_i es la proporción de tiempo que el sistema pasa en el estado i.

PREGUNTA 1 (Modelo del Clima – Generalidades): ¿Puede Q ser una matriz de transición?

No según el PDF, una matriz de transición debe ser una matriz estocástica, lo que exige dos condiciones: "$p_{ij} \geq 0$ for all i, j ∈ I and $\Sigma_j\, p_{ij} = 1$ (i.e. each row of P is a distribution over I)" — entradas no negativas y cada fila sumando 1. Q incumple ambas:

1) Entrada negativa: $q_{33} = -0.2 < 0$. Cada entrada q_{ij} es una probabilidad de transición, y una probabilidad no puede ser negativa.

2) Filas que no suman 1:

- Fila 1: 0.2 + 0.5 + 0.4 = 1.1 ≠ 1 ✗
- Fila 2: 0.1 + 0.5 + 0.3 = 0.9 ≠ 1 ✗
- Fila 3: 0.7 + 0.5 − 0.2 = 1.0 ✓ (pero contiene la entrada negativa)

Cada fila debe ser una distribución de probabilidad sobre los estados: la fila i da las probabilidades de pasar del estado i a cada estado j, y esas probabilidades deben ser no

negativas y sumar exactamente 1. Ninguna fila de Q cumple ambas condiciones a la vez, por lo que Q no puede ser matriz de transición de un proceso de Markov.

PREGUNTA 2 (Modelo del Clima – Generalidades): ¿Puede alguno de los vectores ser el vector de estado actual?

No, ninguno. Según el PDF, el vector de estado es una distribución de probabilidad sobre los estados: un vector fila λ con $\lambda_i \geq 0$ para todo i y $\Sigma_i \lambda_i = 1$.

Ambos vectores tienen entradas no negativas ✓, pero fallan la normalización:

- $u_0 = (0.3, 0.6, 0.7) \rightarrow$ suma = 0.3 + 0.6 + 0.7 = 1.6 ≠ 1 ✗
- $w_0 = (0.2, 0.1, 0.5) \rightarrow$ suma = 0.2 + 0.1 + 0.5 = 0.8 ≠ 1 ✗

Las componentes del vector de estado son las probabilidades de que el sistema esté en cada estado (Soleado, Nublado, Lluvioso), y como esos tres estados agotan todas las posibilidades, sus probabilidades deben sumar exactamente 1. Con u_0 "sobra" probabilidad (1.6) y con w_0 "falta" (0.8), por lo que ninguno puede ser el vector de estado actual de un proceso de Markov.

(5 pts) MODELO DEL CLIMA — EJERCICIO

Datos:

$$P = \begin{bmatrix} 0.2 & 0.5 & 0.3 \\ 0.1 & 0.6 & 0.3 \\ 0.5 & 0.3 & 0.2 \end{bmatrix}$$

v_0 = (0.75, 0.20, 0.05) [estados: Soleado, Nublado, Lluvioso]
Regla de evolución: v_{n+1} = v_n · P

1. **El vector de estado para mañana v = (P(S), P(N), P(L)).**

v_1 = v_0 · P:

P(S) = 0.75(0.2) + 0.20(0.1) + 0.05(0.5) = 0.150 + 0.020 + 0.025 = 0.195
P(N) = 0.75(0.5) + 0.20(0.6) + 0.05(0.3) = 0.375 + 0.120 + 0.015 = 0.510
P(L) = 0.75(0.3) + 0.20(0.3) + 0.05(0.2) = 0.225 + 0.060 + 0.010 = 0.295

v_1 = (0.195, 0.510, 0.295) [suma = 1.000 ✓]

2. **Los vectores de estado para pasado mañana y de hoy a tres días.**
   v_2 = v_1 · P:

   P(S) = 0.195(0.2) + 0.510(0.1) + 0.295(0.5) = 0.0390 + 0.0510 + 0.1475 = 0.2375
   P(N) = 0.195(0.5) + 0.510(0.6) + 0.295(0.3) = 0.0975 + 0.3060 + 0.0885 = 0.4920
   P(L) = 0.195(0.3) + 0.510(0.3) + 0.295(0.2) = 0.0585 + 0.1530 + 0.0590 = 0.2705

   v_2 = (0.2375, 0.4920, 0.2705) [suma = 1.000 ✓]

   v_3 = v_2 · P:

   P(S) = 0.2375(0.2) + 0.4920(0.1) + 0.2705(0.5) = 0.0475 + 0.0492 + 0.1353 = 0.2320
   P(N) = 0.2375(0.5) + 0.4920(0.6) + 0.2705(0.3) = 0.1188 + 0.2952 + 0.0812 = 0.4952
   P(L) = 0.2375(0.3) + 0.4920(0.3) + 0.2705(0.2) = 0.0713 + 0.1476 + 0.0541 = 0.2730

   v_3 = (0.2320, 0.4952, 0.2730) [suma ≈ 1.000 ✓]

Nota: ya en v_3 se ve la convergencia hacia el estado estacionario (abajo).
En solo 3 dias la distribución inicial de 75% sol casi desaparece:

v_0 = (0.7500, 0.2000, 0.0500)
v_1 = (0.1950, 0.5100, 0.2950)
v_2 = (0.2375, 0.4920, 0.2705)
v_3 = (0.2320, 0.4952, 0.2730)
π ≈ (0.2323, 0.4949, 0.2727) ← límite

3. Calcule el estado estacionario de la matriz de transición P.

1. Buscamos un vector π=(x,y,z) que no cambie al aplicar la matriz: π·P = π

2. La matriz es: |0.2 0.5 0.3| |0.1 0.6 0.3| |0.5 0.3 0.2|

3. Al multiplicar π·P se obtiene: 0.2x+0.1y+0.5z=x 0.5x+0.6y+0.3z=y 0.3x+0.3y+0.2z=z Además: x+y+z=1

4. Primera ecuación: 0.2x+0.1y+0.5z=x -0.8x+0.1y+0.5z=0 ×10 → -8x+y+5z=0 y=8x-5z

5. Segunda ecuación: 0.5x+0.6y+0.3z=y 0.5x-0.4y+0.3z=0 ×10 → 5x-4y+3z=0 Sustituyendo y: 5x-4(8x-5z)+3z=0 5x-32x+20z+3z=0 -27x+23z=0 z=27/23 x

6. Hallar y: y=8x-5(27/23)x y=49/23 x

7. Hallar x: x+(49/23)x+(27/23)x=1 x(99/23)=1 x=23/99

8. Hallar y y z: y=49/99 z=27/99

RESULTADO: π=(23/99,49/99,27/99) ≈(0.2323,0.4949,0.2727)

Interpretación: A largo plazo el sistema se estabiliza en: - Soleado: 23.2% - Nublado: 49.5% - Lluvioso: 27.3%

4. ¿Cuál debería ser el vector de estado actual para que mañana el vector de estado sea
v = (0.75, 0.20, 0.05)?

Buscamos $x=(x_1,x_2,x_3)$ tal que $x \cdot P = (0.75, 0.20, 0.05)$.
Sistema:

$0.2x_1 + 0.1x_2 + 0.5x_3 = 0.75$
$0.5x_1 + 0.6x_2 + 0.3x_3 = 0.20$
$x_1 + x_2 + x_3 = 1$

Resolviendo (sustituyendo $x_3=1-x_1-x_2$ y reduciendo el sistema), se obtiene:
x = (-3.5, 2.0, 2.5)
Esta solución no es válida como vector de estado, ya que sus componentes no cumplen $0 \leq x_i \leq 1$ (x_1 es negativo, x_2 y x_3 superan 1).
Conclusión: no existe ningún vector de estado válido hoy que produzca ese resultado mañana. La razón es estructural: P(Soleado mañana) = $0.2x_1+0.1x_2+0.5x_3$ es un promedio ponderado de (0.2, 0.1, 0.5), por lo que nunca puede superar 0.5 (su valor máximo, alcanzado si todo el peso está en Lluvioso, $p_{31}=0.5$). Como se pide P(S)=0.75 > 0.5, es imposible con esta matriz P.

5. Si la entrada ij de la matriz P la denotamos por pij, donde los índices i, j corresponden a los estados i y j respectivamente (1 para soleado, 2 para nublado y 3 para lluvioso),

a) ¿qué representa la probabilidad pij en términos de los estados i y j?
$p_{ij} = P(X_{n+1} = j \mid X_n = i)$: es la probabilidad condicional de que mañana el sistema esté en el estado j, dado que hoy está en el estado i. En este modelo, la probabilidad de que mañana el clima sea j sabiendo que hoy es i. La fila i completa da la distribución del clima de mañana condicionada al clima de hoy, y por eso cada fila suma 1.

b) Si S_0, N_0 y L_0 corresponden a los estados actuales, y S, N y L a los estados de mañana, identifique con las entradas de la matriz P las siguientes probabilidades:

	S(j=1)	N(j=2)	L(j=3)
S (i=1)	0.2	0.5	0.3
N (i=2)	0.1	0.6	0.3
L (i=3)	0.5	0.3	0.2

i) $P(S|S_0) = p_{11} = 0.2$
Probabilidad de que mañana esté soleado dado que hoy lo está (fila 1, columna 1).

ii) $P(N|L_0) = p_{32} = 0.3$
Probabilidad de que mañana esté nublado dado que hoy llueve (fila 3: hoy lluvia; columna 2: mañana nublado).

iii) $P(L|N_0) = p_{23} = 0.3$
Probabilidad de que mañana llueva dado que hoy está nublado (fila 2: hoy nublado; columna 3: mañana lluvia).

TENSORES – RESUMEN DEL ARTÍCULO
"A Simple and Efficient Tensor Calculus for Machine Learning" — Laue, Mitterreiter y Giesen (arXiv 2010.03313, 2020)

El problema.
TensorFlow, PyTorch, JAX y autograd son muy buenos calculando gradientes, pero pídeles un Jacobiano o un Hessiano y se caen: varios órdenes de magnitud más lentos. Y sin Hessianos no hay optimización de segundo orden. Los mismos autores ya lo habían resuelto en NeurIPS 2018, pero con notación de Ricci — incompatible con estos frameworks, que trabajan en notación de Einstein (lo mismo que usa numpy.einsum). Este paper cierra ese gap.

La propuesta.
Definen un solo operador de multiplicación tensorial que cubre producto interno, externo y Hadamard de una vez. En simple: en vez de tres reglas distintas de derivación, una sola que se aplica de forma sistemática sobre el DAG de la expresión. Con eso montan la diferenciación automática en tres modos: forward, reverse (que para funciones escalares es literalmente el backprop de siempre) y cross-country, que combina ambos.

Resultado clave: compresión de Hessianos.
Ahí está el truco: el primer tensor parcial de la cadena siempre resulta ser unitario. Si reordenas las multiplicaciones (cross-country lo permite), eliminas los productos triviales por la identidad y un Hessiano de cuarto orden se comprime a un tensor de segundo o tercero. El impacto no es marginal: en factorización de matrices (n=1000, k=10) pasas de ~1 segundo a ~10 microsegundos. 100,000x de mejora.

Experimentos.
Benchmarks en CPU y GPU con regresión logística, factorización matricial y redes neuronales. Para gradientes todos los frameworks empatan — backprop ya es óptimo, ahí no había nada que ganar. Para Hessianos el método supera a TF, PyTorch, JAX y autograd por varios órdenes de magnitud; JAX directamente se quedó sin memoria con los Hessianos de redes neuronales.
Conclusión.
La verdad es que el mensaje del paper es más grande que el paper: las derivadas de segundo orden dejaron de ser un lujo computacional. No hizo falta más hardware, sino mejores matemáticas — entender la estructura del cálculo vale más que la fuerza bruta. Con esto, cosas que eran prohibitivas en deep learning, como los optimizadores tipo Newton o el análisis de curvatura del loss, pasan a ser factibles.